# Ближайшие соседи

## Шпаргалка



Формулы и вызовы из урока «Ближайшие соседи». Строка с формулой — что это, формула, ниже пример. Строка с кодом — что ищут, что делает, ниже пример и после стрелки его значение, снятое запуском. Примеры кода — на этих массивах:

```
import numpy as np
X = np.array([[1, 2], [2, 0], [4, 4], [5, 2]])
y = np.array([0, 0, 1, 1])
r = np.array([1.0, 2.0, 3.0, 5.0])
q = np.array([2, 2])
a = np.array([0, 1, 1, 0])
b = np.array([0, 1, 0, 0])
```

## Данные и качество

**Доля верных и MSE** — $\text{acc} = \frac{1}{n}\sum_i [a(x_i) = y_i]$, $\text{MSE} = \frac{1}{n}\sum_i \big(a(x_i) - y_i\big)^2$

▷ ответы $(0,1,1,0)$ при верных $(0,1,0,0)$: $\text{acc} = \frac{3}{4}$; ответы $(2,4)$ при $(1,4)$: $\text{MSE} = \frac{1}{2}$

**Train и test** — качество на train не говорит о новых объектах; меряют на отложенных, гиперпараметры выбирают скользящим контролем

▷ константа «всегда самый частый класс» — точка отсчёта

**Стандартизация** — $z = (x - \mu)/\sigma$, $\sigma^2 = \frac{1}{n}\sum_i (x_i - \mu)^2$; $\mu$ и $\sigma$ — по train

▷ train $1, 3, 5$: $\mu = 3$, $\sigma^2 = \frac{8}{3}$; test 7 даёт $z = \sqrt{6} \approx 2{,}45$

**Min-max** — $z = (x - m)/(M - m)$, $m$ и $M$ — по train; на test бывает вне $[0, 1]$

▷ train $1, 3, 5$, test 7: $z = \frac{3}{2}$

`(a == b).mean()` — доля верных: ответы `a`, верные `b`

```
(a == b).mean() → 0.75
```

`((a - b) ** 2).mean()` — MSE

```
((a - b) ** 2).mean() → 0.25
```

`X.mean(axis=0)` — средние признаков; разброс — `X.std(axis=0)`

```
X.mean(axis=0) → [3. 2.]
```

`(x - mu) / sd` — стандартизация числами train

```
((q - X.mean(0)) / X.std(0)).round(1) → [-0.6  0. ]
```

## Соседи

**Расстояния** — $\rho_2 = \sqrt{\sum_j (x_j - q_j)^2}$, $\rho_1 = \sum_j |x_j - q_j|$

▷ $x = (1, 2)$, $q = (3, 1)$: $\rho_2 = \sqrt{5}$, $\rho_1 = 3$

**$k$ соседей** — класс — самый частый среди $k$ ближайших, регрессия — среднее их $y$; масштаб признаков важен

▷ голоса $2:1$ — первый класс

**Взвешенные** — $\arg\max_c \sum_{i \in N_k(q)} w_i [y_i = c]$, регрессия $\sum w_i y_i / \sum w_i$, например $w_i = 1/\rho(x_i, q)$

▷ соседи на расстояниях 1, 2, 4 с ответами 10, 20, 40: $30 / \frac{7}{4} = \frac{120}{7}$

**Скользящий контроль** — train на $m$ частей, каждая по разу отложена; leave-one-out — $m = n$, объект не сосед самому себе

▷ иначе при $k = 1$ LOO равен 1 на любых данных без повторов

`np.abs(X - q).sum(axis=1)` — L1 до всех объектов

```
np.abs(X - q).sum(axis=1) → [1 2 4 3]
```

`np.linalg.norm` — L2 до всех объектов

```
d = np.linalg.norm(X - q, axis=1)
```

`np.argsort` — номера $k$ ближайших

```
np.argsort(d, kind="stable")[:3] → [0 1 2]
```

`np.bincount` — голоса, `argmax` — самый частый

```
np.bincount(y[[0, 1, 2]]).argmax() → 0
```

`r[nb].mean()` — регрессия соседями

```
r[np.argsort(d)[:3]].mean() → 2.0
```

## Вызовы sklearn

Порядок у всех моделей: создать с гиперпараметрами, `fit(X, y)`, `predict(X)`; у преобразований — `transform`. Выученное — в полях с подчёркиванием. Один признак — столбцом, `x[:, None]`. Модули — `sklearn.preprocessing` и `sklearn.neighbors`.

`StandardScaler` — `fit` по train; поля `mean_`, `scale_`

```
sc = StandardScaler().fit(X)
```

`sc.transform` — те же числа на test

```
sc.transform([[3, 2]]) → [[0. 0.]]
```

`KNeighborsClassifier` — L1: `p=1`; веса: `weights="distance"`

```
knn = KNeighborsClassifier(3).fit(X, y)
```

`knn.predict` — предсказать классы

```
knn.predict([q]) → [0]
```

`knn.kneighbors` — номера соседей, по возрастанию расстояния

```
knn.kneighbors([q])[1] → [[0 1 2]]
```

`KNeighborsRegressor` — `predict` даёт среднее $y$ соседей

```
kr = KNeighborsRegressor(3).fit(X, r)
```

`kr.predict` — предсказать числа

```
kr.predict([q]) → [2.]
```

## Где sklearn расходится с точными правилами

**Ничьи** — при равных голосах — меньший номер класса; при равных расстояниях порядок соседей не обещан и зависит от `algorithm`

▷ сетка $7 \times 7$, вопрос в центре клетки: `"brute"` и `"kd_tree"` дают разные классы

**Стандартизация** — `fit_transform` на test — ошибка: числа берут только по train

▷ `sc.transform(Xt)` после `sc.fit(X)`